



OPERATING SYSTEMS

UE24CS242B

IPC - Shared Memory & Message Passing

Pavan A C

Department of Computer Science and
Engineering

OPERATING SYSTEMS

- ~~Slides Credits for all the PPTs of this course~~
The slides/diagrams in this course are an adaptation, combination, and enhancement of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberchatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating System: Three Easy Pieces, Remzi Arpaci



OPERATING SYSTEMS

I PC - Shared Memory and Message Passing

Pavan A C

Department of Computer Science and Engineering

OPERATING SYSTEMS

Interprocess Communication

- Processes within a system may be *independent* or *cooperating*
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - Shared memory**
 - Message passing**

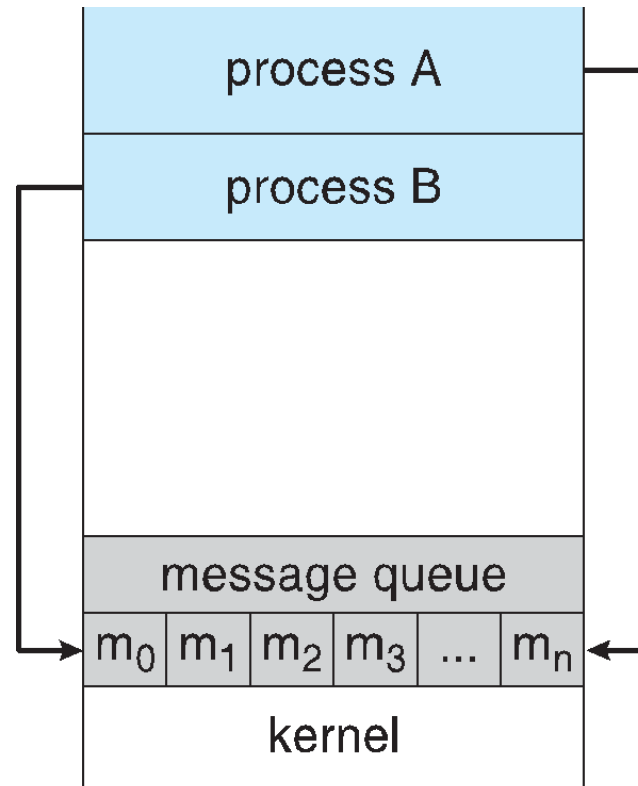


- **Independent** process cannot affect or be affected by the execution of another process
- **Cooperating** process can affect or be affected by the execution of another process
- Advantages of process cooperation
 - Information sharing
 - Computation speed-up
 - Modularity
 - Convenience

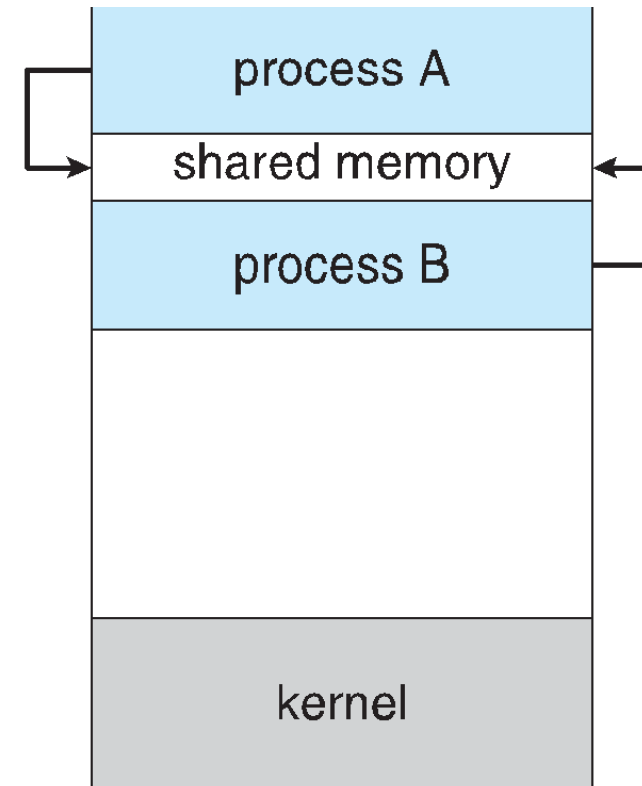
- Two models of IPC

a) Message passing

b) Shared memory



(a)



(b)



- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
 - **unbounded- buffer** places no practical limit on the size of the buffer
 - Consumer may have to wait for new items, but the producer can always produce new items
 - **bounded- buffer** assumes that there is a fixed buffer size
 - Consumer must wait if the buffer is empty; producer must wait if the buffer is full

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item

item buffer[ BUFFER_SIZE ];
int in = 0;
int out = 0;
```
- Shared buffer is implemented as a circular array with 2 logical pointers: **in** and **out**
- Buffer is empty when **in == out**; buffer is full when $((\mathbf{in} + 1) \% \text{BUFFER_SIZE}) == \mathbf{out}$
- Variable **in** points to the next free position in the buffer; **out** points to the first full position in the buffer


```
item next_produced;
while (true) {
    /* produce an item in
next_produced */
    while (((in + 1) %
BUFFER_SIZE) == out)
        ; /* do nothing */

    buffer[in] =
next_produced;
    in = (in + 1) %
BUFFER_SIZE;
}
```

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_
}
}
```

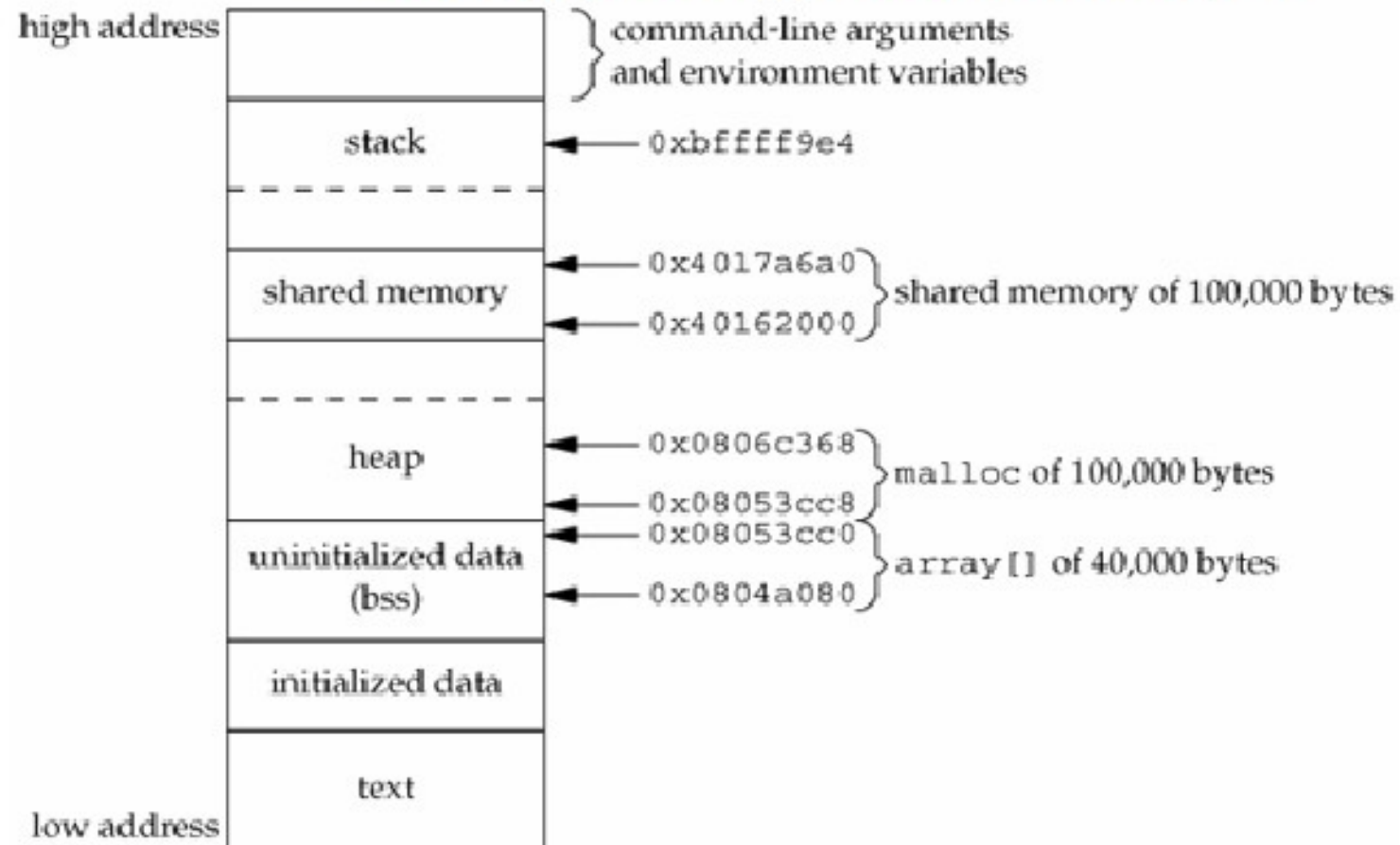
```
item next_consumed;  
while (true) {  
    while (in == out)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
  
    /* consume the item in next_consumed */  
}
```

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the user's processes not the operating system
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.

- Shared memory allows two or more processes to share a given region of memory.
- Shared memory is the fastest form of IPC, because the data does not need to be copied between the client and the server.
- The only trick in using shared memory is synchronizing access to a given region among multiple processes.
- If the server is placing data into a shared memory region, the client shouldn't try to access the data until the server is done.
- Often, semaphores are used to synchronize shared memory access. (record locking can also be used.)



Memory layout on an Intel-based Linux system



- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - **send**(*message*)
 - **receive**(*message*)
- The *message* size is either fixed or variable

- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive
- Implementation issues:
 - How are links established?
 - Can a link be associated with more than two processes?
 - How many links can there be between every pair of communicating processes?
 - What is the capacity of a link?
 - Is the size of a message that the link can accommodate fixed or

- Implementation of communication link
 - Physical :
 - Shared memory
 - Hardware bus
 - Network
 - Logical :
 - Direct or indirect
 - Synchronous or asynchronous
 - Automatic or explicit buffering

- Processes must name each other explicitly:
 - **send** (P , $message$) – send a message to process P
 - **receive**(Q , $message$) – receive a message from process Q
- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional

- Messages are directed and received from mail boxes (also referred to as ports)
 - Each mail box has a unique id
 - Processes can communicate only if they share a mail box
- Properties of communication link
 - Link established only if processes share a common mail box
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional

- Operations
 - create a new mailbox (port)
 - send and receive messages through mailbox
 - destroy a mailbox
- Primitives are defined as:
 - **send**(*A*, *message*) – send a message to mailbox *A*
 - **receive**(*A*, *message*) – receive a message from mailbox *A*

- Mail box sharing
 - P_1 , P_2 , and P_3 share mail box A
 - P_1 sends; P_2 and P_3 receive
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes
 - Allow only one process at a time to execute a receive operation
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
 - **Blocking send** -- the sender is blocked until the message is received
 - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send** -- the sender sends the message and continue
 - **Non-blocking receive** -- the receiver receives:
 - A valid message, or
 - Null message
- Different combinations possible

- Producer - consumer becomes trivial

```
message next_produced;  
while (true) {  
    /* produce an item in next_produced */  
    send(next_produced);  
}
```

```
message next_consumed;  
while (true) {  
    receive(next_consumed);  
  
    /* consume the item in  
    next_consumed */  
}
```

- Queue of messages attached to the link (direct or indirect); messages reside in a temporary queue.
- Queues can be implemented in one of three ways
 1. Zero capacity – no messages are queued on a link.
Sender must wait for receiver (rendezvous)
 2. Bounded capacity – finite length of n messages
Sender must wait if link full
 3. Unbounded capacity – infinite length
Sender never waits
- Zero-capacity case is sometimes referred to as a message system with no buffering; other cases are referred to as systems with automatic



THANK YOU

Pavan A C

Department of Computer Science &
Engineering
pavanac@pes.edu